OWASP Top 10 2025 - Insecure Data Handling

A04: Cryptographic Failures

What are Cryptographic Failures?!

Cryptographic failures happen when sensitive data isn't adequately protected due to lack of encryption, faulty implementation, or insufficient security measures. This includes storing passwords without hashing, using outdated or weak algorithms (such as MD5, SHA1, or DES), exposing encryption keys, or failing to secure data during transmission

How to Prevent Cryptographic Failures

Preventing cryptographic failures starts with choosing strong, modern algorithms and implementing them properly. Sensitive information such as passwords should be hashed using robust, slow hashing functions like bcrypt, scrypt, or Argon2. When encrypting data, avoid creating your own algorithms; instead, rely on trusted, industry-standard libraries.

Practical

TRYHACKME • OWASP TOP 10

Crypto Lab: Weak XOR Cipher

Homegrown encryption with a short key is easily broken.

All three encrypted notes below use the same weak XOR cipher with a 4-character key. Since XOR encryption is reversible and the key is short, you can brute-force it. Enter the correct key once to decrypt all notes at once!

Encrypted Notes

All three notes are encrypted using the same weak XOR cipher with a short key.

Meeting Reminder #1

Encrypted (base64)


```
BiA8RSIrPhE4JjFULzA1VC9lP145ZS1eJiorQyQyeVA/ZWoRGw
h3EQgqN1cuNzxfKCB5QyQqNBEJaw==
```


Decrypted with key "key1"

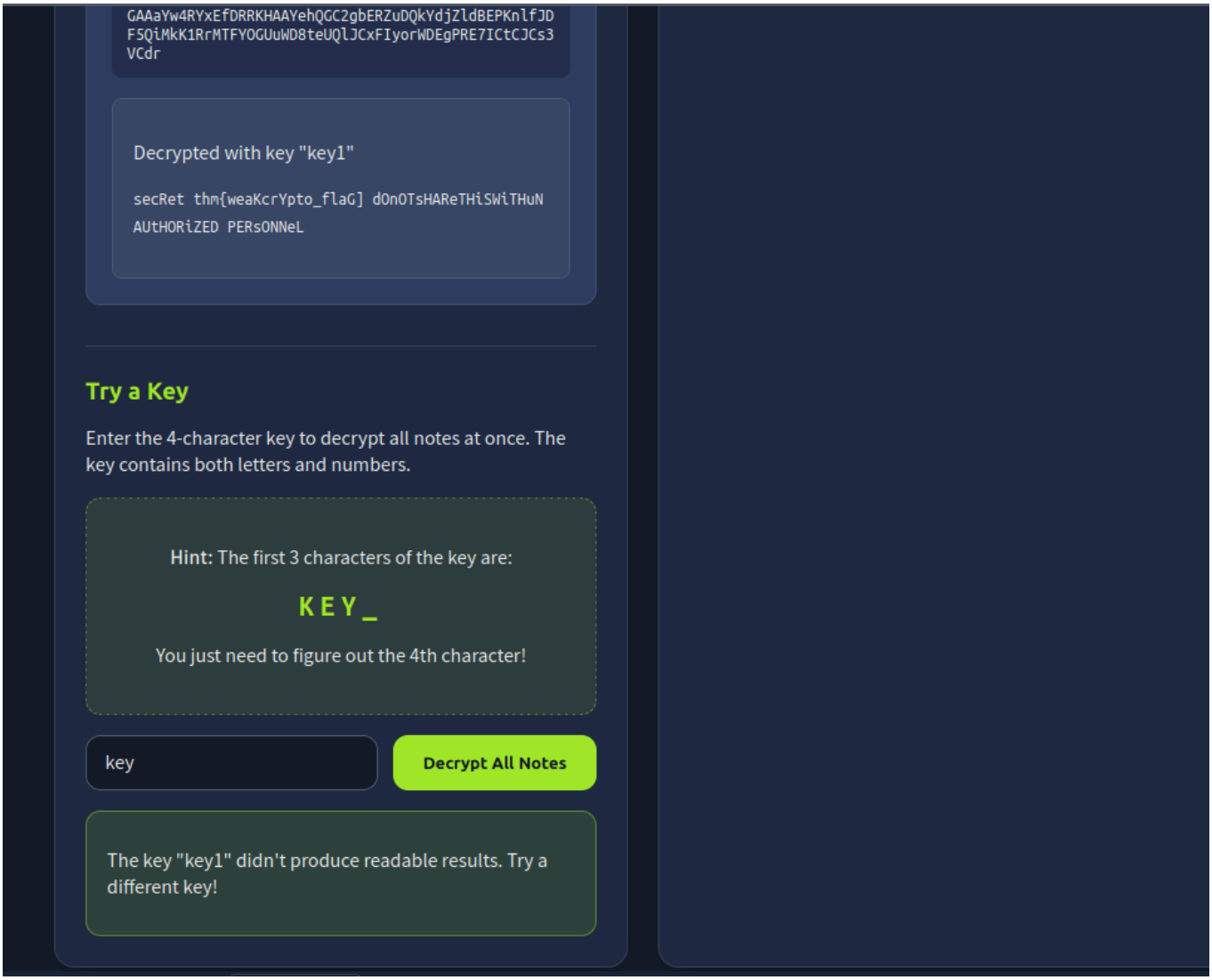
```
mEEtING SCHeDULeDFoRToMORrOWaT pm cONfERENCErOOM
b
```

Why XOR Cipher is Weak

- **Short keys** can be brute-forced quickly (only 4 characters).
- **Repeating key pattern** makes frequency analysis possible.
- **No authentication** - you can't verify if decryption succeeded without context.
- **Homegrown crypto** lacks the security guarantees of proven algorithms.

Key Information:

- The key is exactly 4 characters long
- It contains both letters and numbers
- First 3 characters are shown as a hint above
- One key decrypts all three notes simultaneously



The weak 4 character Xor cipher key allowed me to brute force attack trying different combinations of key

the key that I managed to decrypt with was KEY1 and it gave me the flag

A05: Injection

What is Injection?

Injection occurs when an application takes user input and mishandles it. Instead of processing the input securely, the application passes it directly into a system that can execute commands or queries, such as a database, a shell, a templating engine or API.

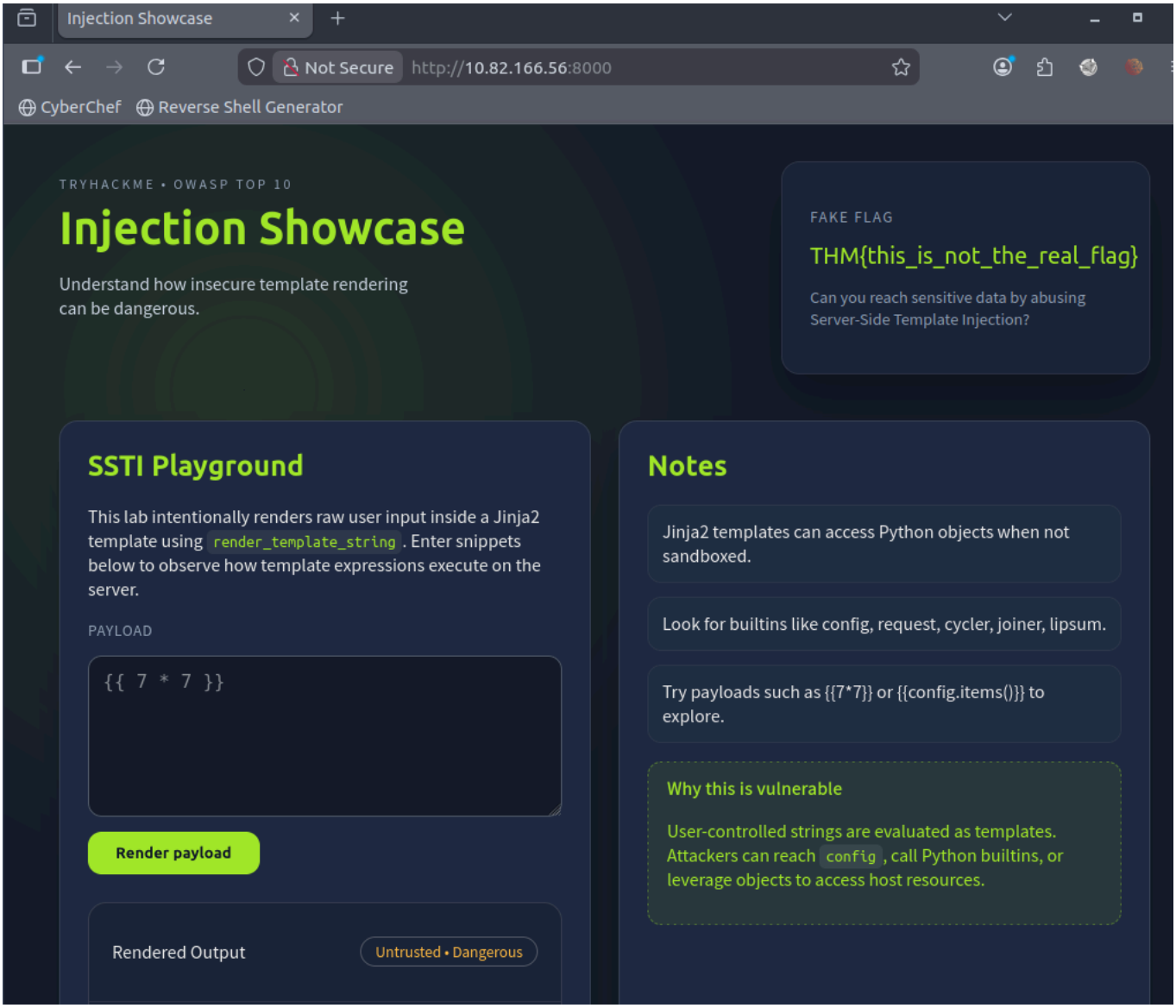
The following are some classic examples of injection that you may be familiar with:

- SQL Injection
- Command Injection
- AI Prompts
- Server Side Template Injection (SSTI)

How to Prevent Injection

Preventing injection starts by ensuring that user input is always treated as untrusted.
Input validation and sanitisation play a crucial role in preventing these types of attack. Escape dangerous characters, enforce strict data types and filter before the application even processes the input.

Practical



can use different commands and will render different output

server.

PAYLOAD

```
{{ config.items()}}
```

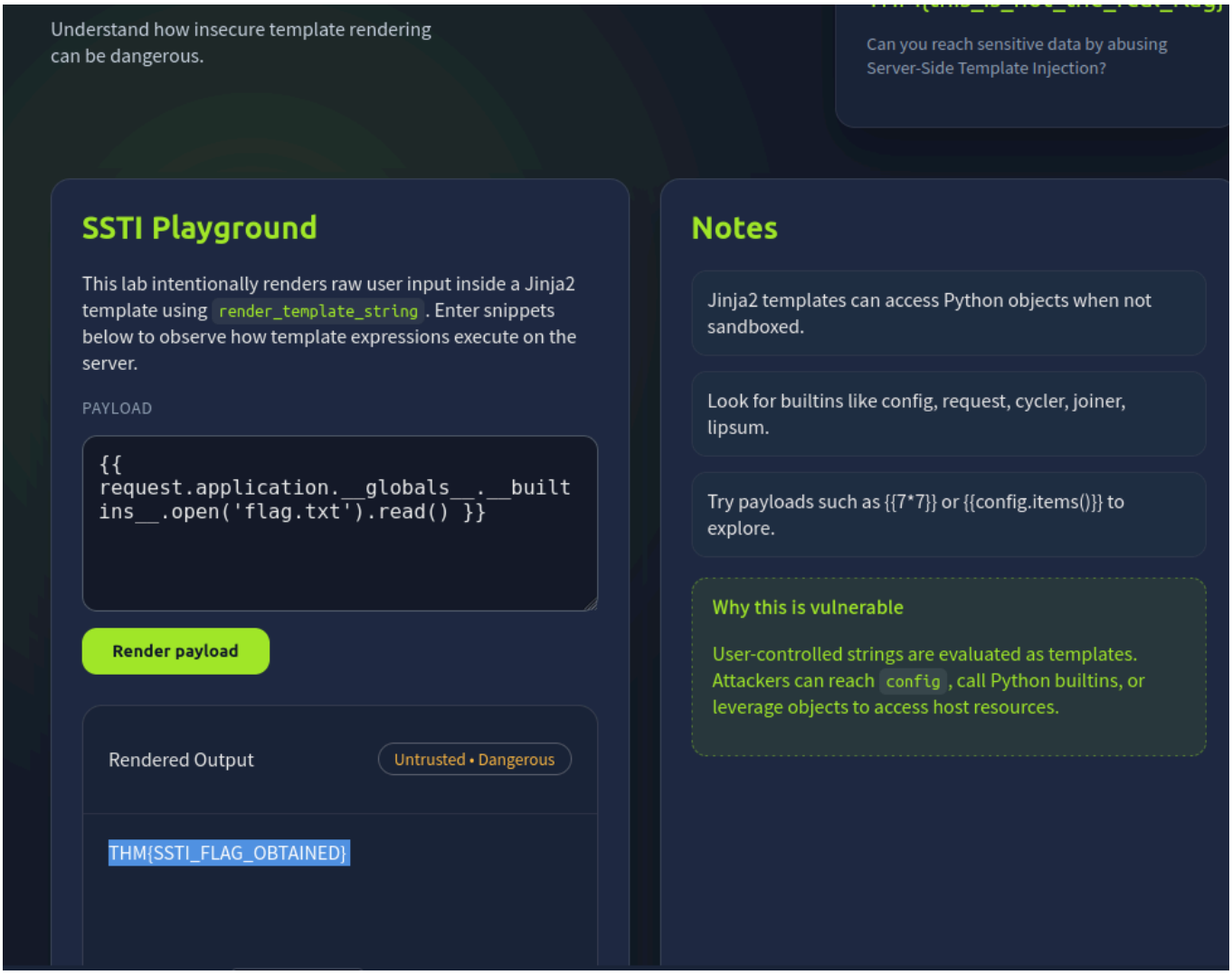
Render payload

Rendered Output

Untrusted • Dangerous

```
dict_items([('DEBUG', True), ('TESTING', False),
('PROPAGATE_EXCEPTIONS', None), ('SECRET_KEY',
None), ('PERMANENT_SESSION_LIFETIME',
datetime.timedelta(days=31)), ('USE_X_SENDFILE',
False), ('SERVER_NAME', None), ('APPLICATION_ROOT',
'/'), ('SESSION_COOKIE_NAME', 'session'),
('SESSION_COOKIE_DOMAIN', None),
('SESSION_COOKIE_PATH', None),
('SESSION_COOKIE_HTTPONLY', True),
('SESSION_COOKIE_SECURE', False),
('SESSION_COOKIE_SAMESITE', None),
('SESSION_REFRESH_EACH_REQUEST', True),
('MAX_CONTENT_LENGTH', None),
('SEND_FILE_MAX_AGE_DEFAULT', None),
('TRAP_BAD_REQUEST_ERRORS', None),
```

the web application uses pythons jinja2 templating to load the page contents.



A08: Software or Data Integrity Failures

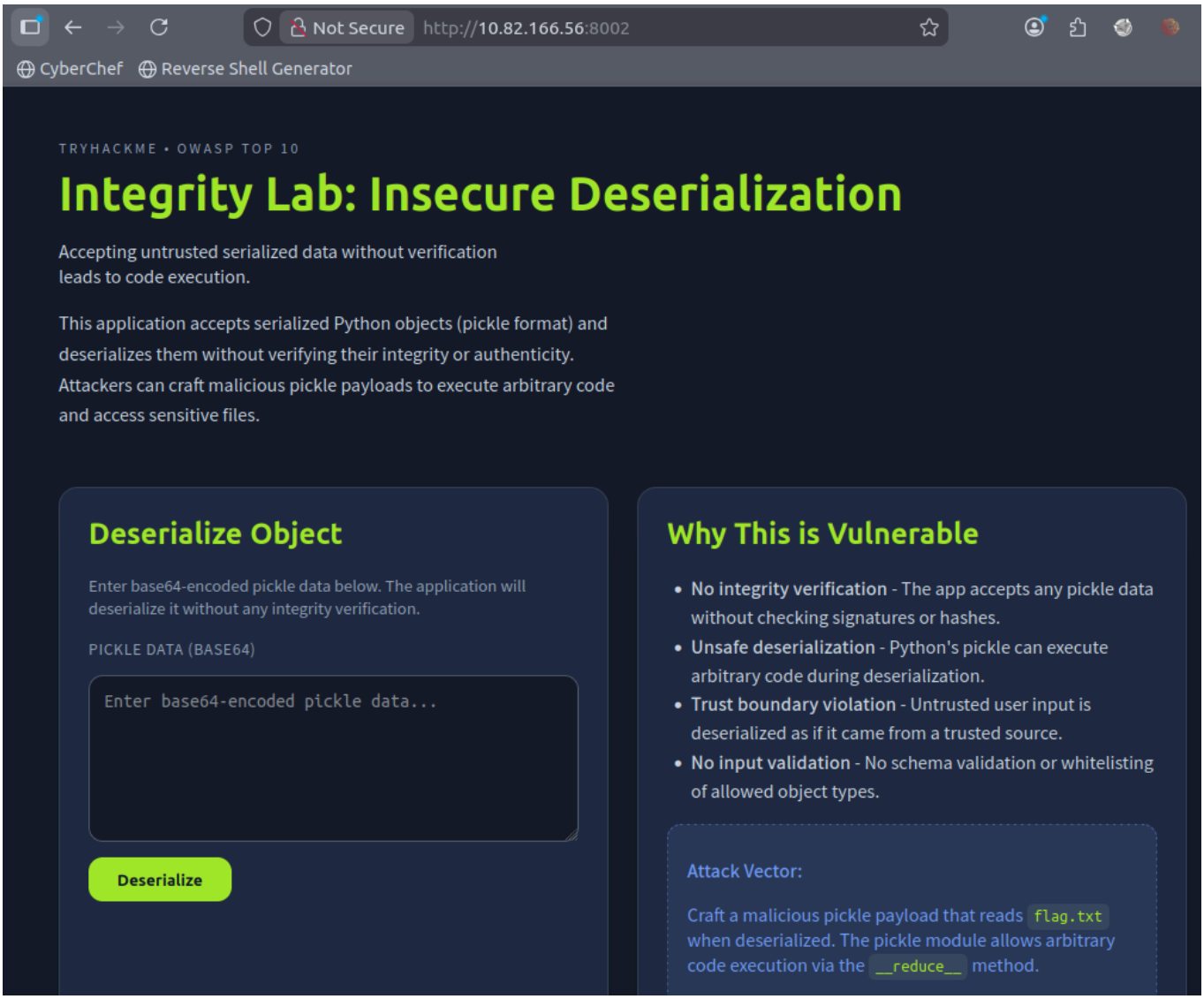
What Are Software or Data Integrity Failures?

Software or Data Integrity Failures occur when an application relies on code, updates, or data it assumes are safe, without verifying their authenticity, integrity, or origin. This includes trusting software updates without verification, loading scripts or configuration files from untrusted sources, failing to validate data that impacts application logic, or accepting data such as binaries, templates, or JSON files without confirming whether it has been altered.

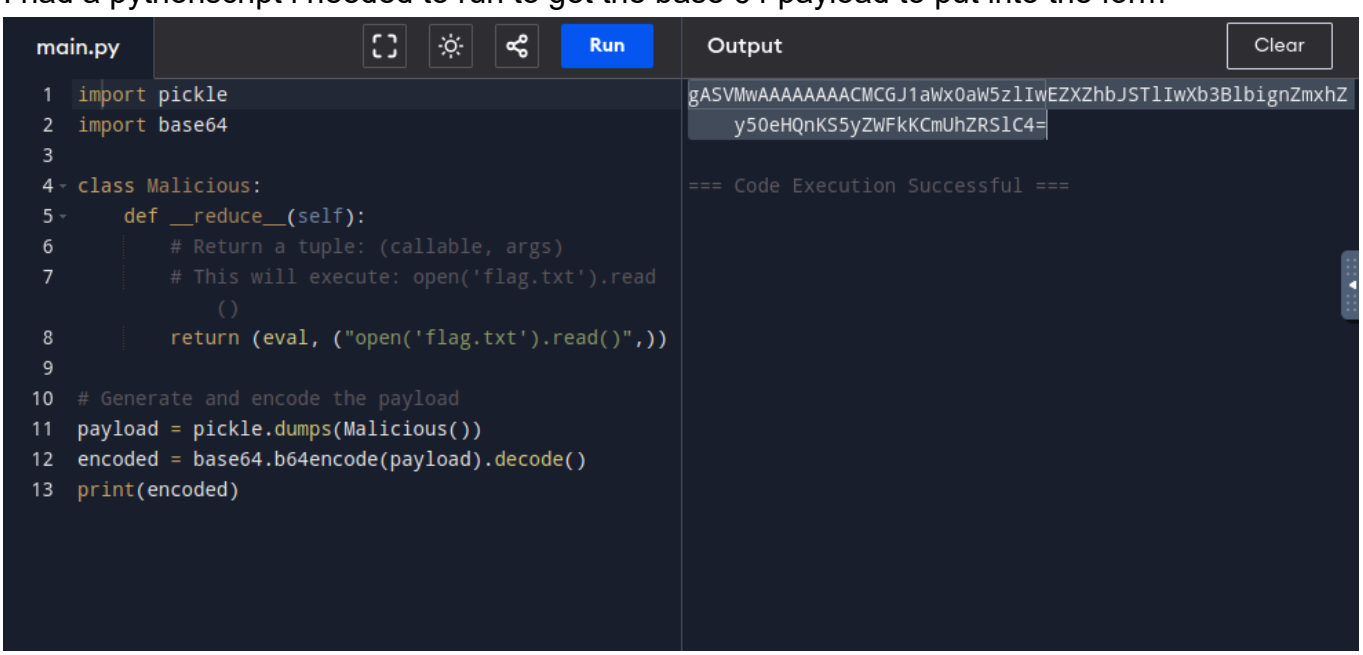
How to Avoid Software & Data Integrity Failures

Preventing these failures begins with establishing trust boundaries. Applications should never assume that code, updates, or key pieces of data are legitimate and automatically trusted; their integrity must be verified. This involves using methods such as cryptographic checks (like checksums) for update packages and ensuring that only trusted sources can modify critical artefacts.

Practical



I had a pythonscript i needed to run to get the base 64 payload to put into the form



i then put the payload in and submitted the form and it produced the flag

Deserialize Object

Enter base64-encoded pickle data below. The application will deserialize it without any integrity verification.

PICKLE DATA (BASE64)

gASVMwAAAAAAAACMCGJ1aWx0aW5zlIwEZXZhbJSTlIwXb3Bl
bignZmxhZy50eHQnKS5yZWFkKCmUhZRSlC4=

Deserialize

Deserialized Output

THM{INSECURE_DESERIALIZATION}